

# Inteligência Artificial

## Busca Competitiva

Ricardo Araújo Rios

Lab.IA  
Instituto de Computação - UFBA



1

## Agenda

- Introdução
- Decisões ótimas em jogos
- Decisões ótimas em jogos com vários participantes
- Poda alfa-beta



2



## Objetivo

- Conhecer métodos de busca competitiva utilizadas em teoria dos jogos.

## Introdução

- Até o momento vimos:
  - Problemas sem interação com outro agente;
  - O agente possui total controle sobre suas ações;
  - O agente possui total controle sobre o efeito de suas ações;
  - Muitas vezes encontra a solução ótima.
- Mas...



# Introdução

- E se:
  - O ambiente for multiagente?
  - As ações de outros agentes afetam o agente em ação?
  - As metas dos agentes estão em conflito?
  - Não procura-se por um caminho, mas um movimento?



# Introdução

- Os melhores movimentos de um agente dependem dos movimentos de outro agente (adversário)
- Teoria dos jogos:
  - Visualiza qualquer ambiente multiagente como um jogo
  - Se o impacto de cada agente sobre os outros for significativo, não importa se são competitivos ou cooperativos.



## Busca competitiva!



7

## Busca competitiva!

Os melhores movimentos de um agente dependem dos movimentos de outro agente (adversário)

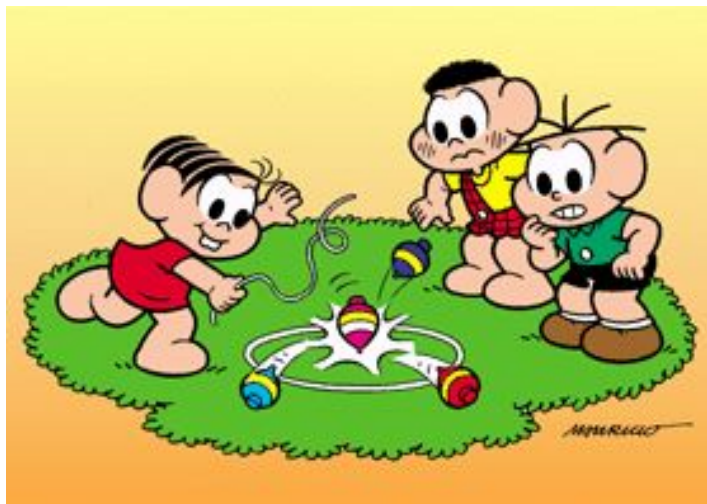
8



## Busca competitiva

- As metas dos agentes estão em conflito
- O oponente é “imprevisível”
- É preciso tomar uma decisão, mesmo que não seja ótima (limite de tempo)
- Problema que usa essa abordagem: JOGOS

## Jogos com Adversários



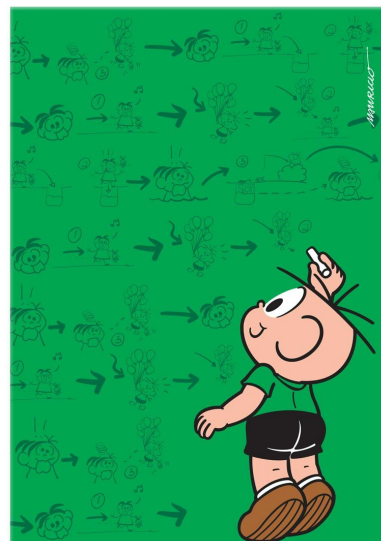


## Jogos x Problemas de busca

- Em jogos, o comportamento do oponente é imprevisível
  - Solução é uma estratégia (política)
  - Formulação simples (ações bem definidas)
  - Em geral, os agentes estão restritos a um pequeno número de ações
  - Resultados definidos por regras precisas
  - É possível medir o sucesso ou fracasso

## Jogos x Problemas de busca

- É preciso definir um plano de ataque



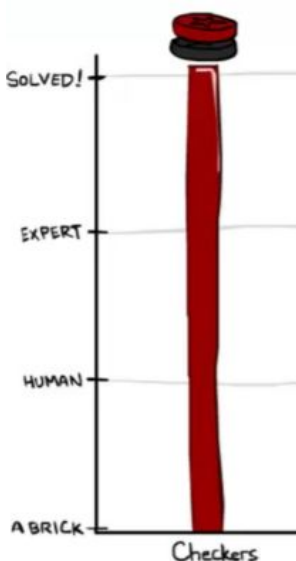


## Busca competitiva - *two-player zero-sum games*

- Considere:
  - Dois jogadores se revezando
  - A vitória de um jogador significa a derrota do outro
  - Ambientes determinísticos e totalmente observáveis
- Exemplos:
  - Jogo-da-velha, damas, xadrez, Go
- Soluções:
  - Algoritmo Minimax e Poda Alfa-Beta

13

## Busca competitiva



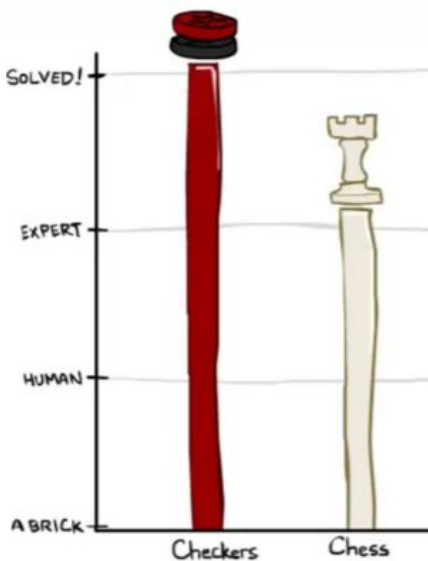
Fonte: [ai.berkeley.edu](http://ai.berkeley.edu)

- Damas (Checkers) - O primeiro jogador de damas de computador foi criado em 1950. Desde então, as damas se tornaram um jogo **resolvido**, o que significa que qualquer posição pode ser avaliada como vitória, derrota ou empate deterministicamente para ambos os lados, dado que ambos os jogadores atuam de forma otimizada.



14

# Busca competitiva

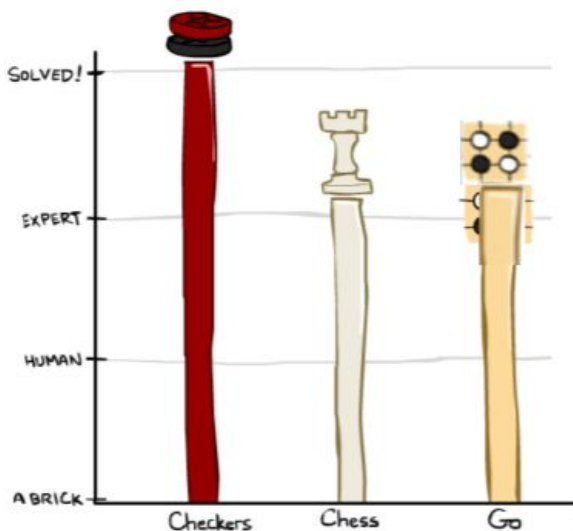


Fonte: [ai.berkeley.edu](http://ai.berkeley.edu)

- Xadrez (Chess) - Em 1997, o Deep Blue se tornou o primeiro agente de computador a derrotar o campeão de xadrez humano Gary Kasparov em uma partida de seis jogos. O Deep Blue foi construído para usar métodos extremamente sofisticados para avaliar mais de 200 milhões de posições por segundo. Os programas atuais são ainda melhores, embora menos históricos. Resolvido, mas não há garantia de quem vencerá será o preto ou branco.



# Busca competitiva

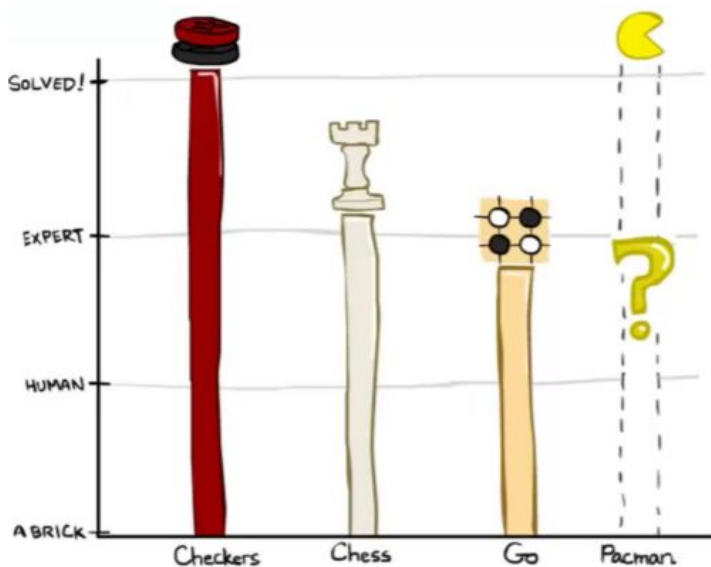


Fonte: [ai.berkeley.edu](http://ai.berkeley.edu)

- Go - O espaço de busca para Go é muito maior do que para xadrez, então a maioria não acreditava que os agentes do Go derrotariam campeões mundiais humanos. No entanto, AlphaGo, desenvolvido pelo Google, derrotou historicamente o campeão de Go Lee Sodol por 4 jogos a 1 em março de 2016.



# Busca competitiva



Fonte: ai.berkeley.edu

*Todos esses agentes campeões mundiais usam, pelo menos até certo ponto, as técnicas de busca com adversários que vamos estudar hoje.*

## Busca competitiva

- Jogadores se revezam até o jogo acabar
  - $S_0 \rightarrow$  Estado Inicial
  - Jogador(s)  $\rightarrow$  Define a vez de qual jogador no estado  $s$
  - Resultado( $s,a$ )  $\rightarrow$  Modelo de transição
  - Teste( $s$ )  $\rightarrow$  Teste de término
  - Utilidade( $s,p$ )  $\rightarrow$  compensação do jogador  $p$  no estado terminal  $s$ .



## Busca competitiva

- Espaço de estados pode ser grande demais
  - Jogo da Velha: 362880 nós
  - Xadrez:  $10^4$  nós
- Árvore de busca: árvore que está sobreposta ao espaço de estados, examinando os nós suficientes para permitir que um jogador determine que jogada fazer



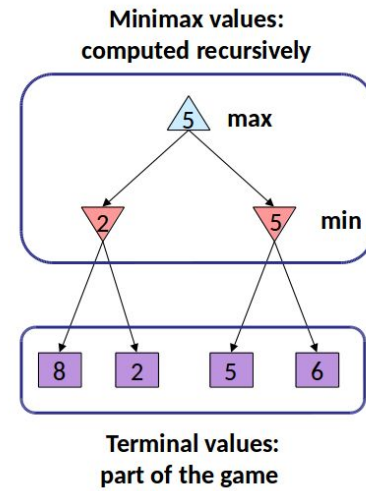
## MINIMAX

- Considere um jogo com 2 jogadores
- Eles alternam jogadas até o jogo acabar
- Um jogador maximiza a utilidade (MAX)
- Outro jogador minimiza a utilidade (MIN)
- Análogo ao algoritmo E-OU de busca



# MINIMAX

- Jogos determinísticos e de soma zero
  - Jogo da Velha, xadrez, dama



# MINIMAX

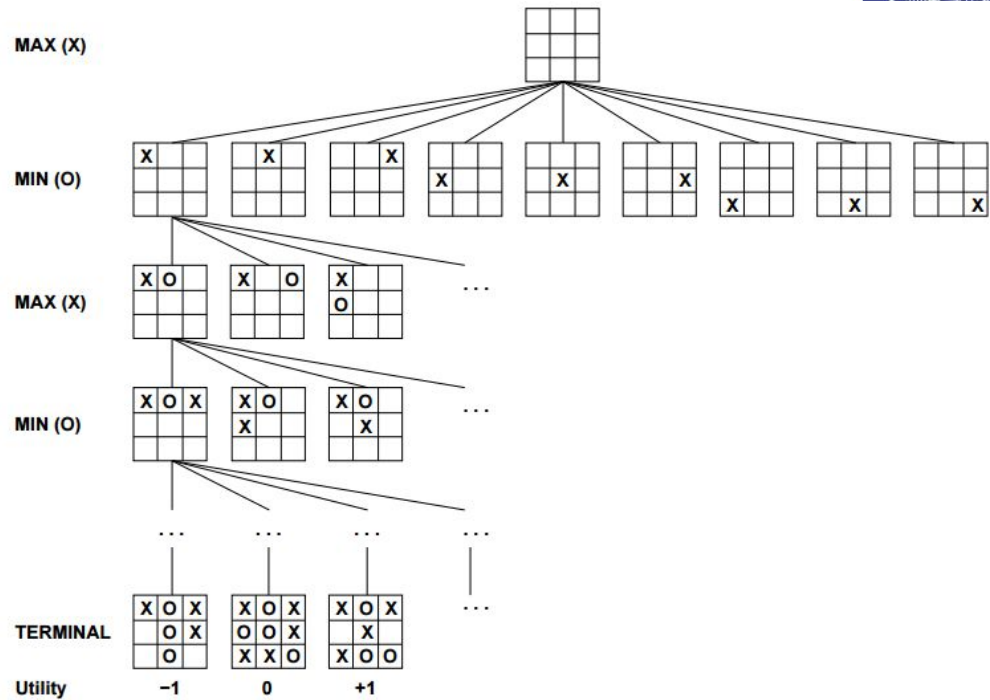
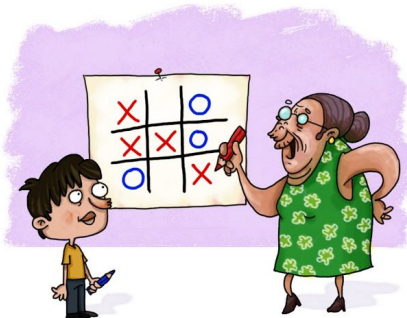
- Jogo da velha
  - MAX(**X**)
  - MIN(**0**)





# MINIMAX

- Jogo da velha
  - MAX(X)
  - MIN(O)

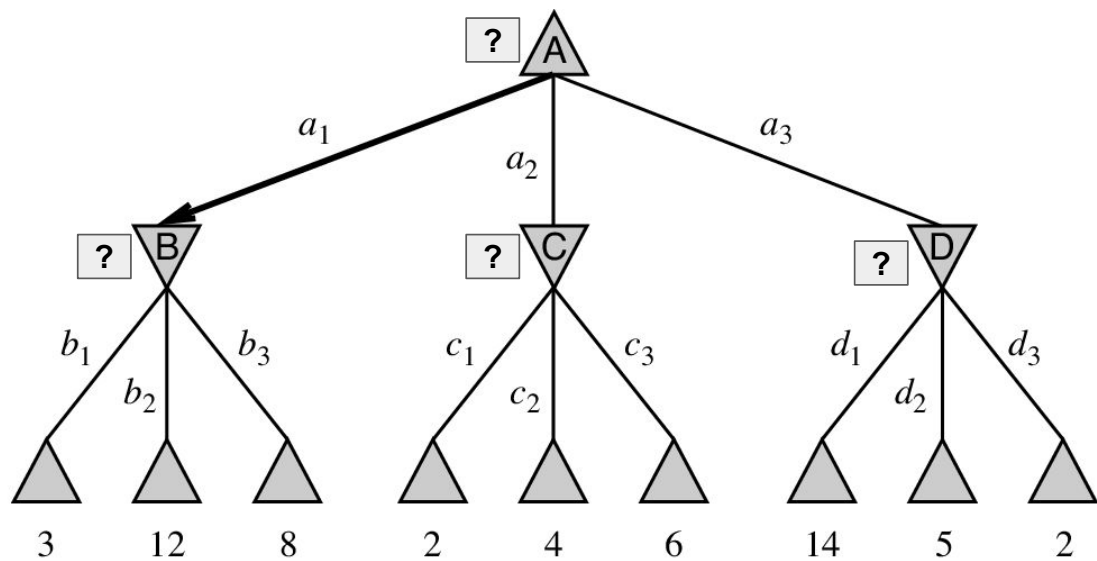


## Algoritmo MINIMAX



MAX

MIN

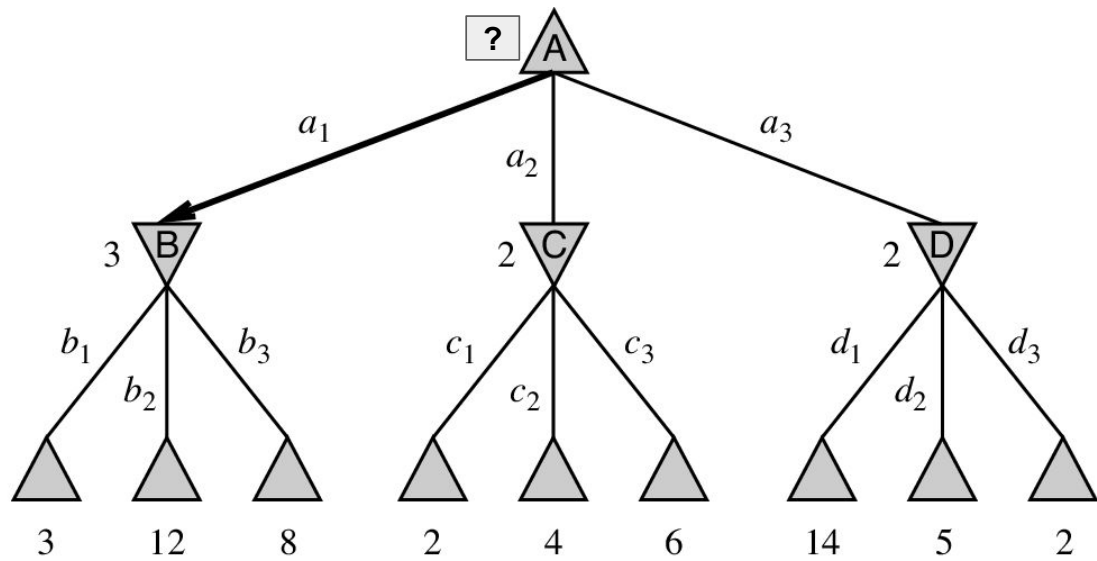


# Algoritmo MINIMAX



MAX

MIN



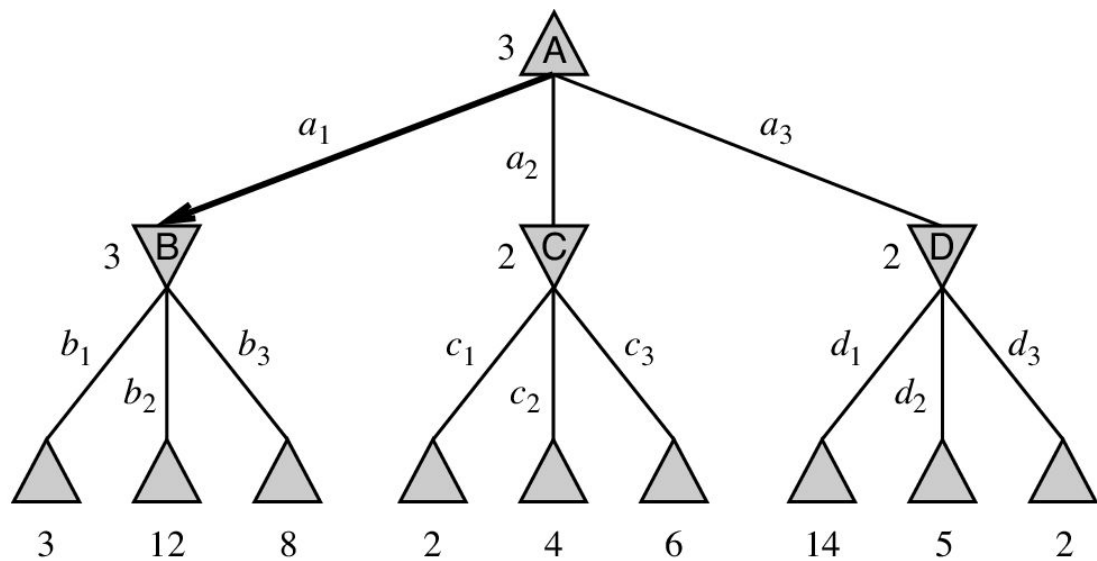
25

# Algoritmo MINIMAX



MAX

MIN

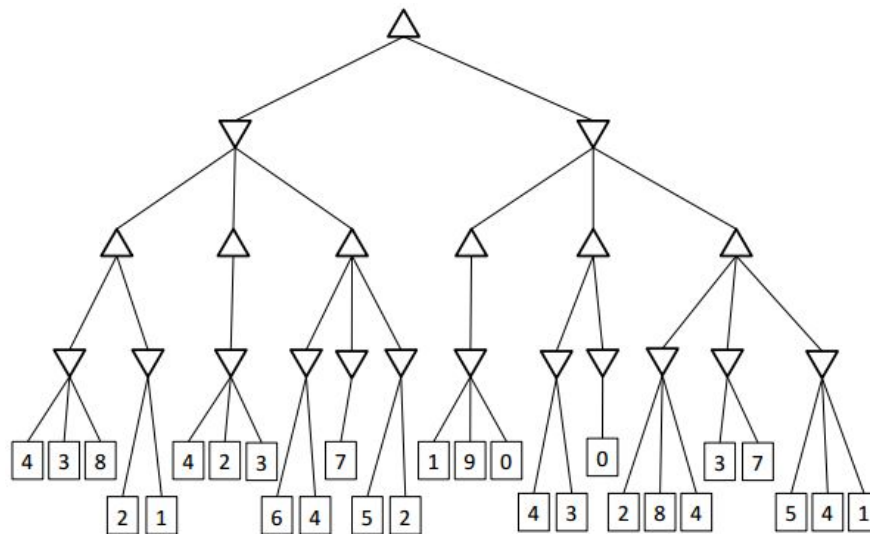


26



## Exercício 1

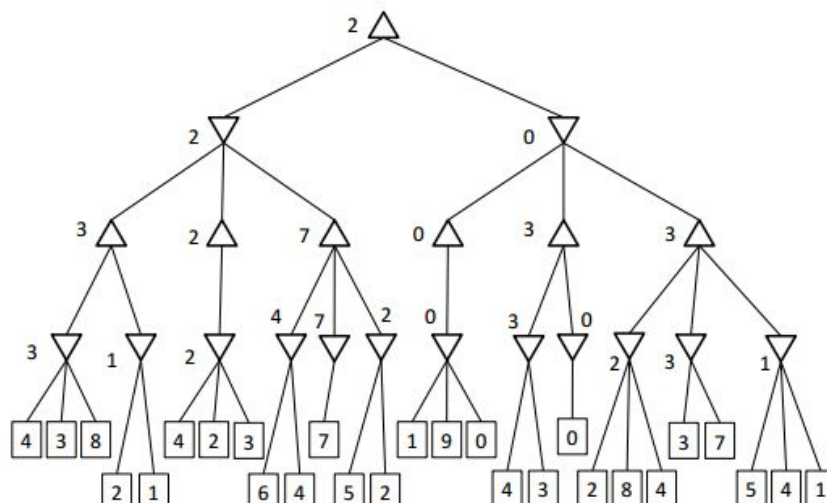
Apresente os valores de cada estado encontrados pelo minimax.



27

## Exercício 1 - Resposta

Apresente os valores de cada estado encontrados pelo minimax.



28



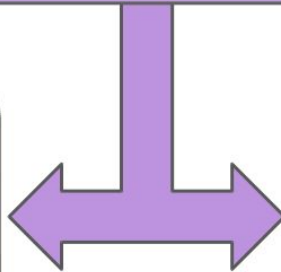


# Minimax

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$



```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

29

## Busca competitiva

- Algoritmo Minimax
  - Para implementar este algoritmo, vamos formalizar a noção de **terminal utilities** e **state values**
  - O valor de um estado é a pontuação ideal atingível pelo agente que controla aquele estado.
  - Para ter uma noção do que isso significa, observe o seguinte tabuleiro de jogo Pacman trivialmente simples:

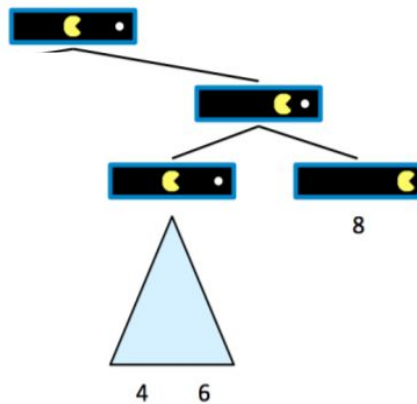


30



## Busca competitiva

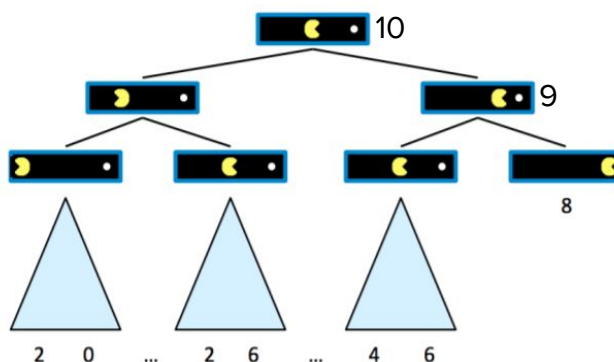
- Algoritmo Minimax
  - Suponha que Pacman **comece com 10 pontos e perca 1 ponto por movimento até comer a bolinha**, momento em que o jogo chega a um estado final e termina.



31

## Busca competitiva

- Algoritmo Minimax
  - Podemos começar a construir uma árvore de jogo (*game tree*) para este tabuleiro da seguinte maneira, onde os filhos de um estado são estados sucessores, assim como nas árvores de busca para problemas de busca comuns:



32

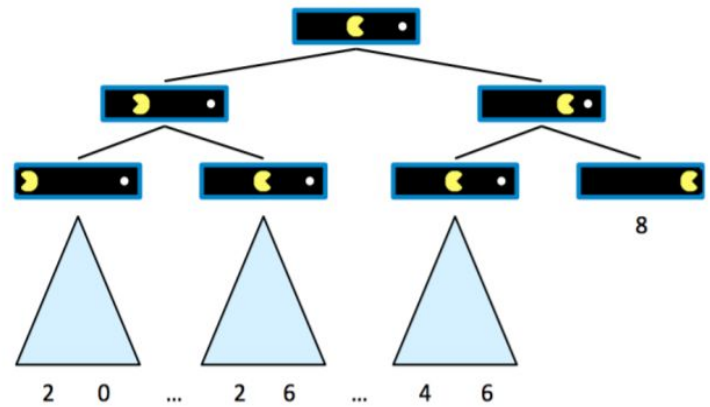




## Busca competitiva

- Algoritmo Minimax

- É evidente a partir desta árvore que se Pacman for direto para a bolinha (comida), ele termina o jogo com uma pontuação de 8 pontos.
- Ao passo que, se ele voltar atrás em qualquer ponto, ele acaba com alguma pontuação de valor inferior.



## Busca competitiva

- Algoritmo Minimax

- O **valor de um estado** não terminal (**utility**) é definido como o melhor resultado possível que um agente pode alcançar a partir desse estado.
- O **valor de um estado final** (**terminal utility**) é sempre algum valor conhecido determinístico e uma propriedade de jogo inerente.
- No exemplo do Pacman, o valor do estado final mais à direita é simplesmente 8, a pontuação que Pacman obtém indo direto para a comida.
- Além disso, no exemplo, o valor de um estado não terminal é definido como o máximo dos valores de seus filhos.





## Busca competitiva

- Algoritmo Minimax
  - Vamos agora apresentar um novo tabuleiro de jogo com um fantasma adversário que quer impedir que Pacman coma a comida.



## Busca competitiva

- Algoritmo Minimax
  - Considere que os dois agentes se revezam nas jogadas, levando a uma árvore de jogo onde os dois agentes desligam as camadas da árvore que eles "controlam".
  - Um agente com controle sobre um nó simplesmente significa que o nó corresponde a um estado onde é a vez do agente e, portanto, é a sua oportunidade de decidir sobre uma ação e alterar o estado do jogo de acordo.

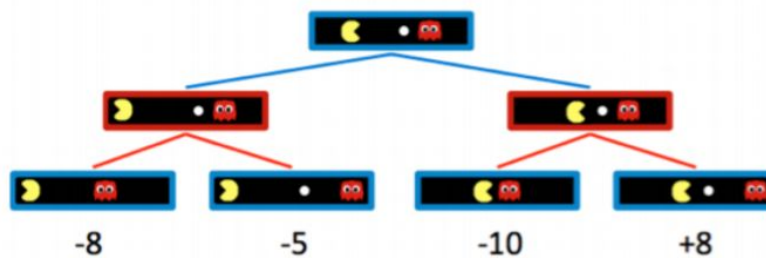




# Busca competitiva

- Algoritmo Minimax

- Observe que todos os filhos de nós controlados por fantasmas são nós onde o fantasma moveu-se para a esquerda ou direita de seu estado no pai e vice-versa para nós controlados pelo Pacman.
- Observe esta árvore simplificada:

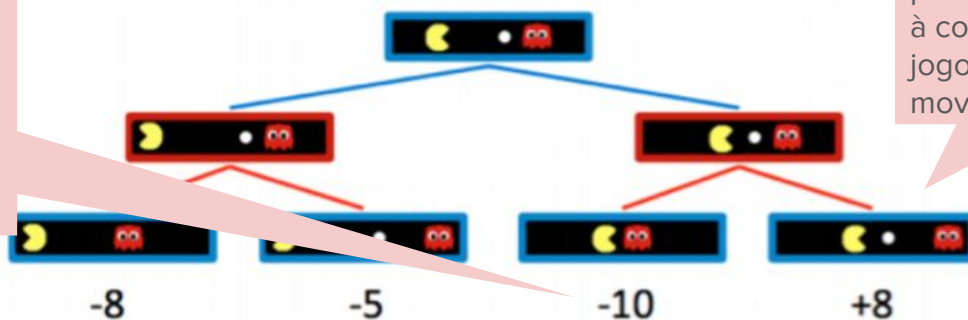


37

# Busca competitiva



Se o fantasma mover-se para a esquerda e o pacman for de encontro à comida, ele perderá o jogo (-10).



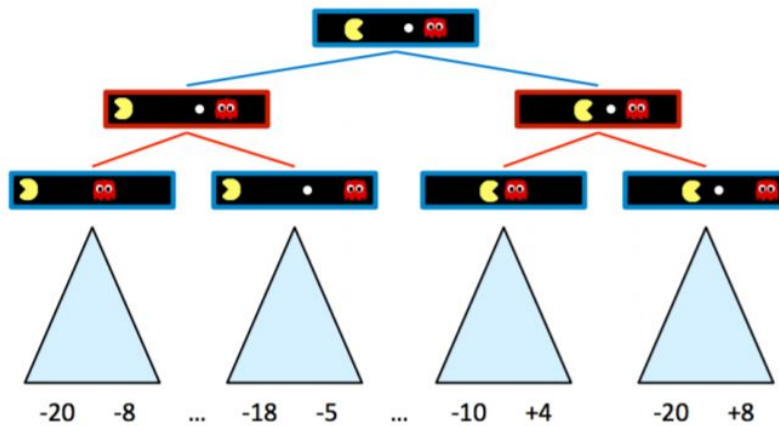
Se o fantasma mover-se para a direita e o pacman for de encontro à comida, ele ganhará o jogo após dois movimentos ( $10-2=8$ ).

38



# Busca competitiva

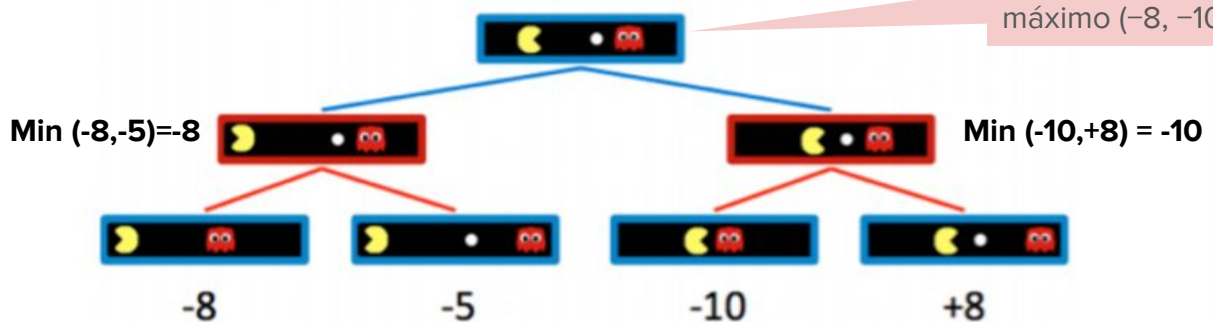
- Algoritmo Minimax
  - Supondo várias jogadas:



39

# Busca competitiva

- Algoritmo Minimax
  - Já que Pacman quer **maximizar** sua pontuação, ele irá para a esquerda e obterá a pontuação de  $-8$  em vez de tentar ir para a comida e pontuar  $-10$



40



# Busca competitiva

- Algoritmo Minimax

- Embora Pacman queira a pontuação de +8 que ele pode obter se ele acaba no nós filho mais à direita, por meio do minimax ele "sabe" que um fantasma com desempenho ideal não permitirá que ele obtenha sucesso.
- Para agir de forma otimizada, Pacman é forçado a proteger suas apostas e de forma contra-intuitiva afaste-se da comida para minimizar a magnitude de sua derrota.

# Busca competitiva

```
function MINIMAX-SEARCH(game, state) returns an action
  player  $\leftarrow$  game.TO-MOVE(state)
  value, move  $\leftarrow$  MAX-VALUE(game, state)
  return move
```

```
function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow$   $-\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move  $\leftarrow$  v2, a
  return v, move
```

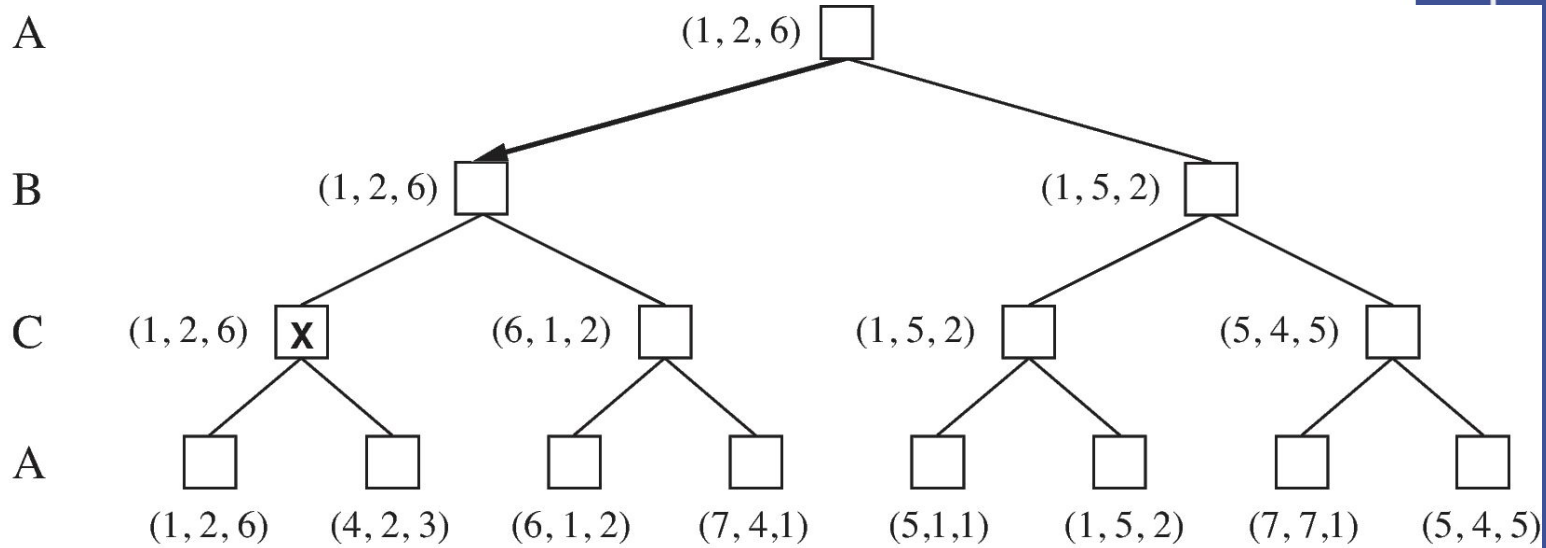
```
function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow$   $+\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move  $\leftarrow$  v2, a
  return v, move
```

As funções MAX-VALUE e MIN-VALUE percorrem toda a árvore do jogo, até as folhas, para determinar o valor de um estado.



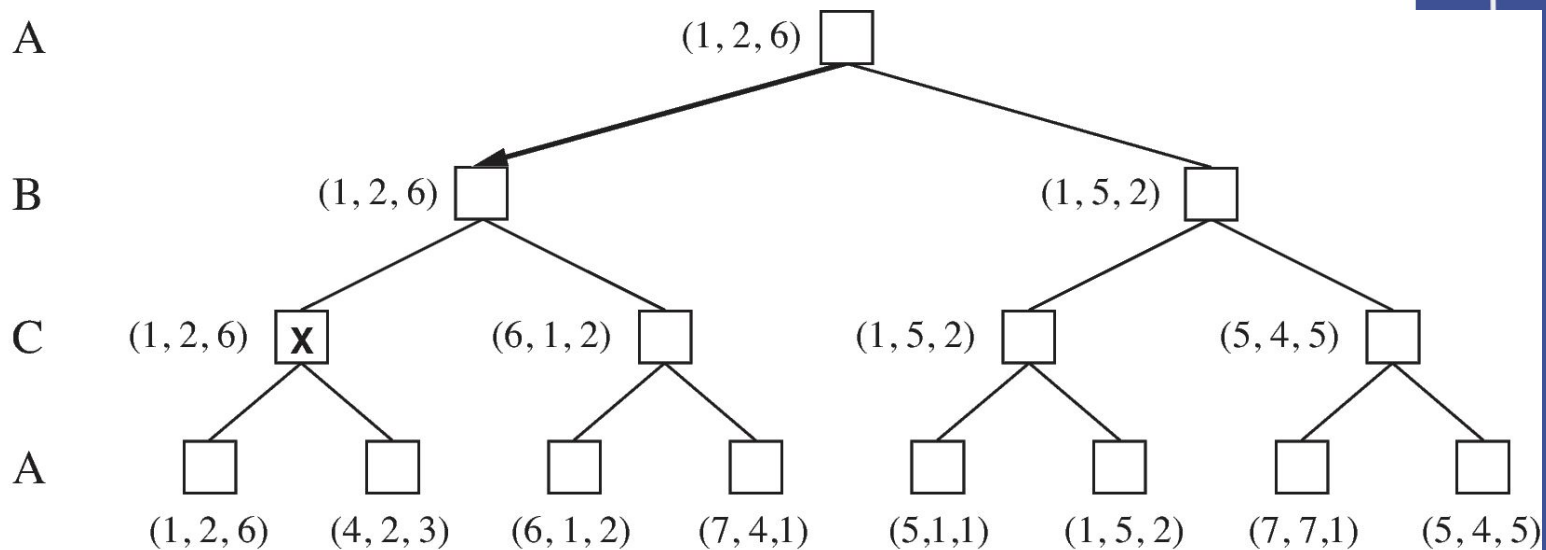


## Jogos com vários participantes



43

## Jogos com vários participantes



- Aliança vs Colaboração

44



## Algoritmo MINIMAX - desempenho

- Completo: sim, se a árvore for finita
- Ótimo: sim
- Complexidade de Tempo:  $(b^m)$  (busca em profundidade)
- Complexidade de Espaço:  $O(b^m)$ 
  - Para o jogo da velha:  $9! = 362.880$  nós
  - Para xadrez:  $10^{40}$  nós



## Algoritmo MINIMAX - Problemas

- Tempo gasto para determinar a solução ótima pode ser impraticável para muitos problemas reais (percorrer a árvore inteira – todas as folhas)
- Porém, Minimax traz solução ótima e pode ser modificado para gerar métodos mais eficientes:

Ex: Podar a árvore e evitar subárvores irrelevantes: **Poda alfa-beta**



## Poda alfa-beta

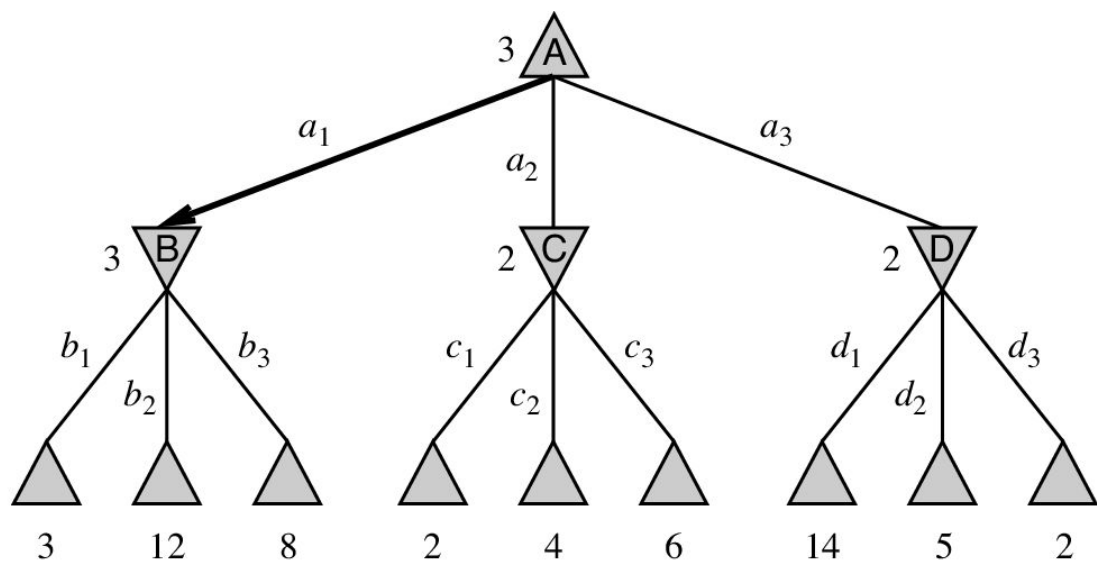
- Calcular a decisão minimax correta sem examinar todos os nós na árvore de jogo
- Não pesquisar um ramo da árvore que comprovadamente não pode levar a um resultado melhor que o atual
- **alfa**: o valor da melhor escolha (mais alto), até o momento, ao longo do caminho para MAX.
- **beta**: o valor da melhor escolha (mais baixo), até o momento, ao longo do caminho para MIN.

## Algoritmo MINIMAX

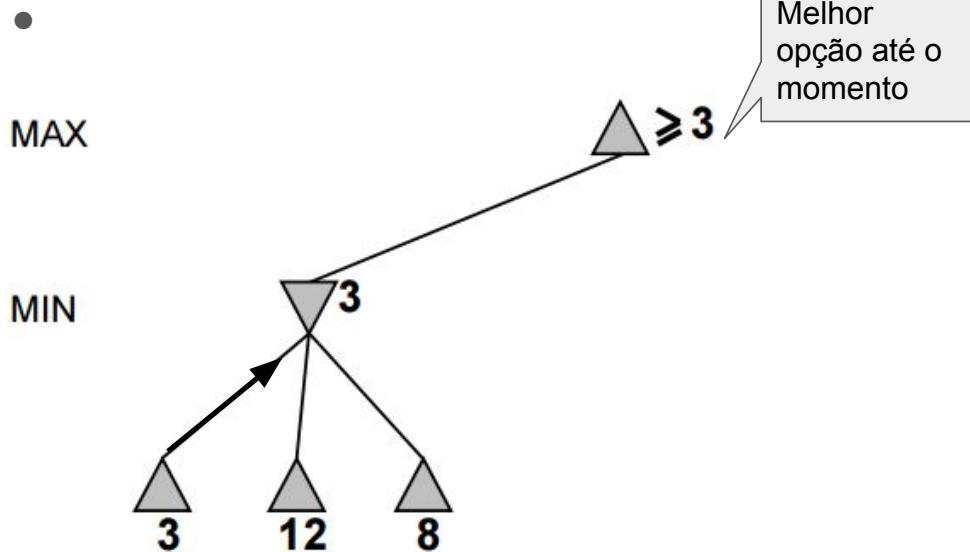


MAX

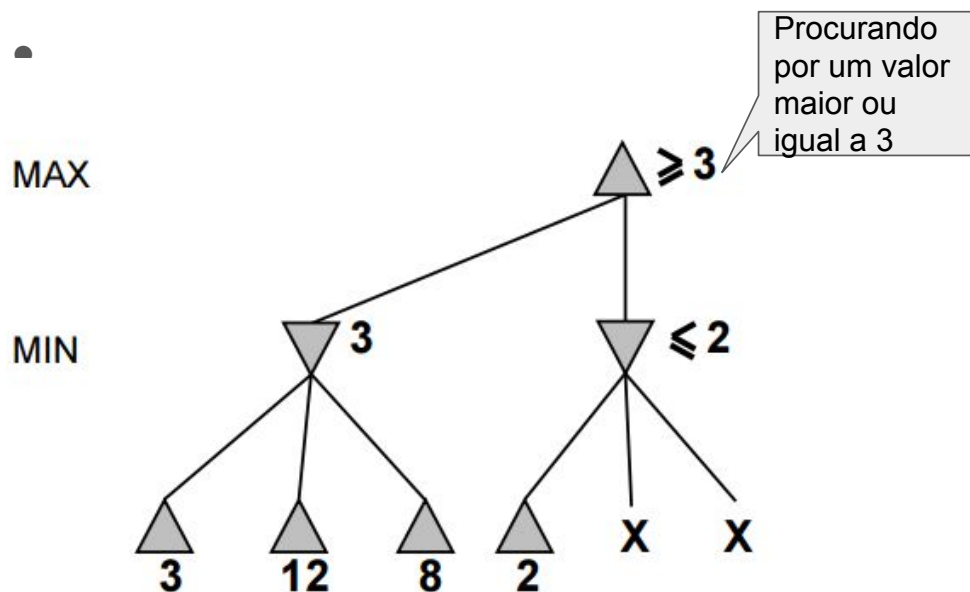
MIN



## Poda alfa-beta



## Poda alfa-beta



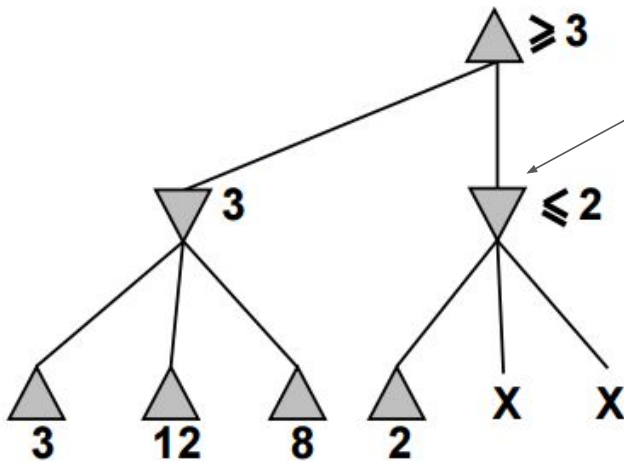


## Poda alfa-beta

•

MAX

MIN



Não é preciso expandir os demais nós, pois:

Se houver um filho MAIOR, ele não será escolhido, pois beta não pode aumentar

Se houver um filho MENOR, ele pode até ser escolhido, mas não vai afetar alfa, que já é 3. Ou seja, sempre que for verificado um beta menor que o alfa já escolhido, tenta outro ramo.

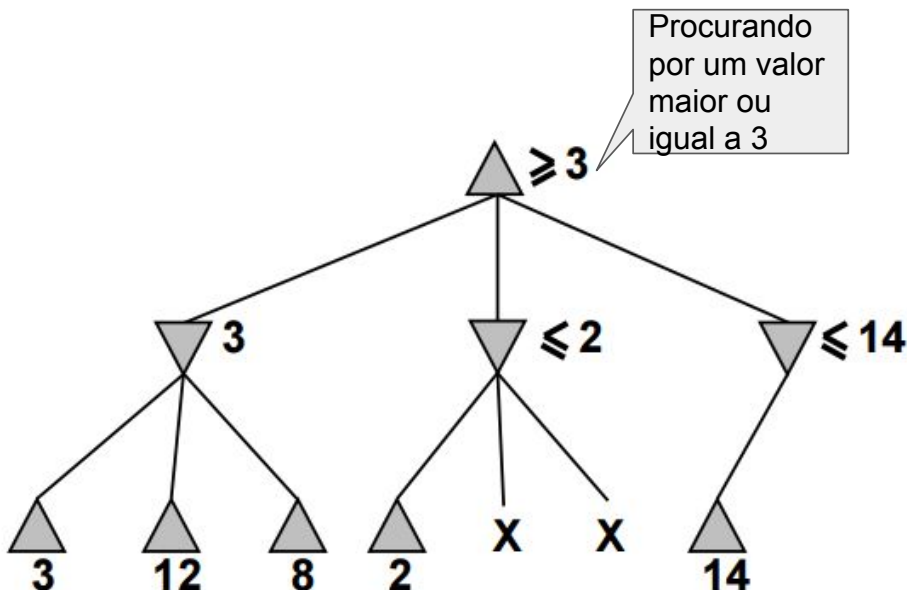
51

## Poda alfa-beta

•

MAX

MIN



52

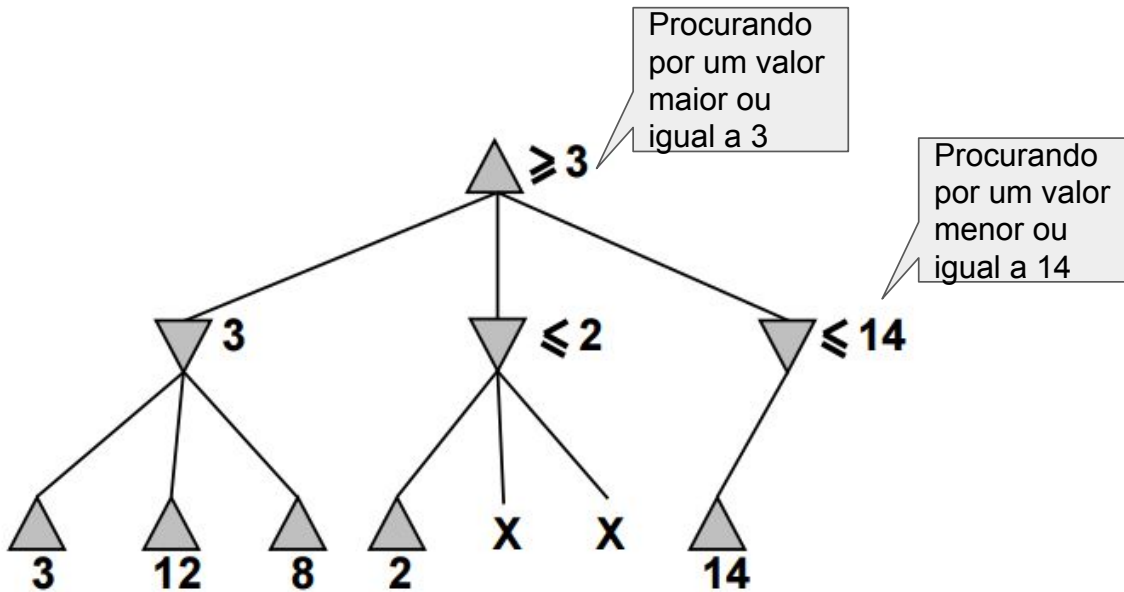


## Poda alfa-beta

•

MAX

MIN



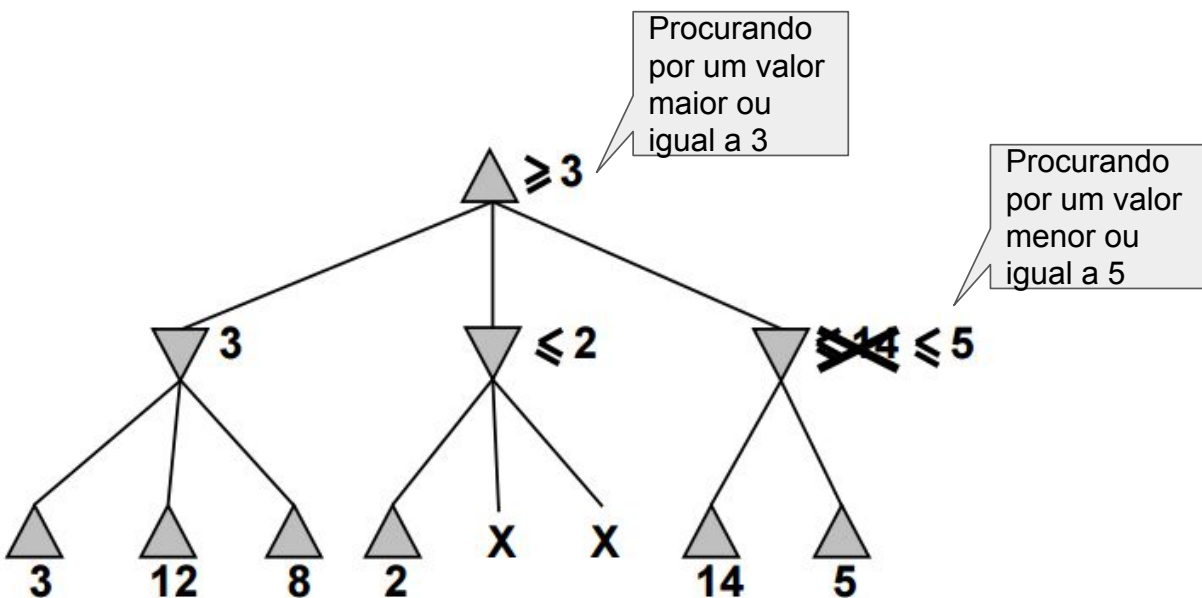
53

## Poda alfa-beta

•

MAX

MIN



54

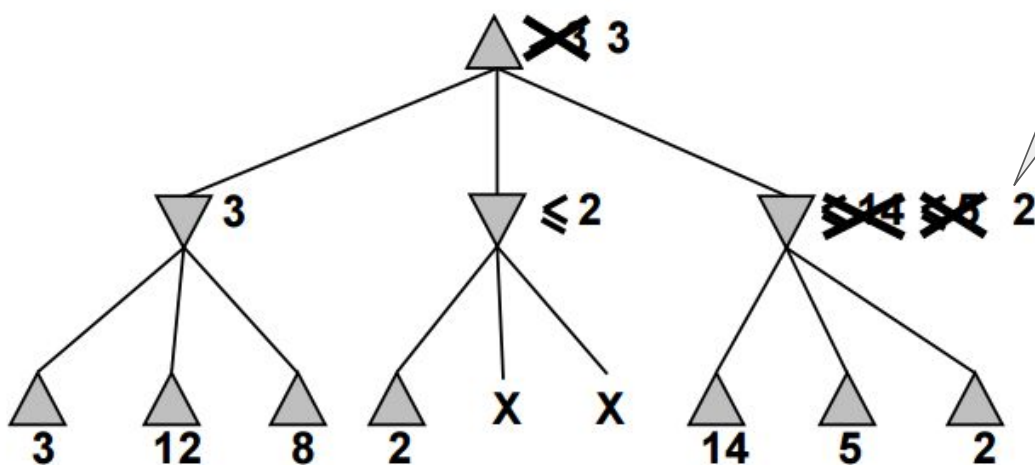


## Poda alfa-beta

•

MAX

MIN



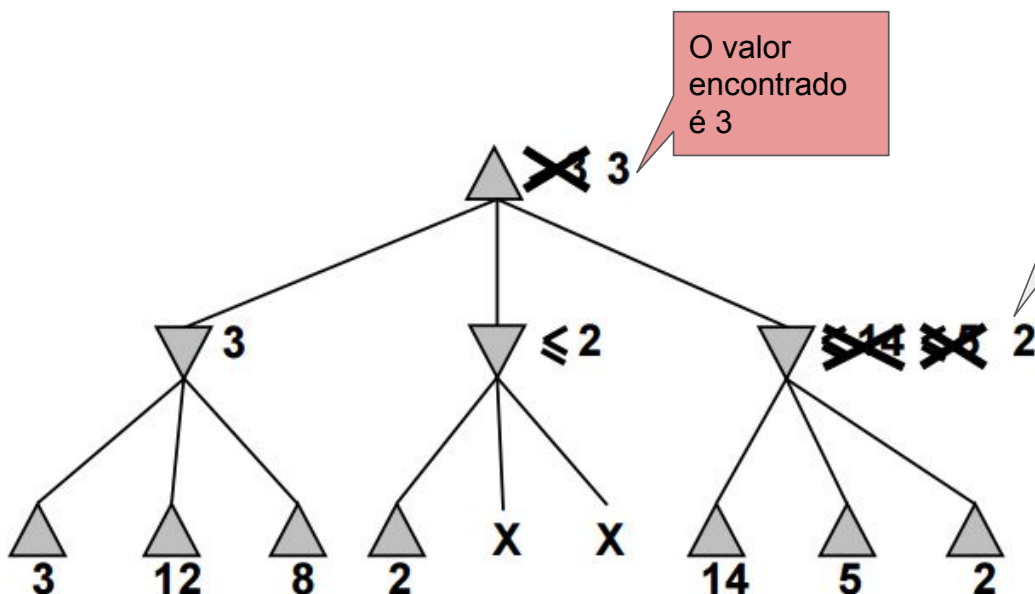
2 não é maior ou igual a 3

## Poda alfa-beta

•

MAX

MIN



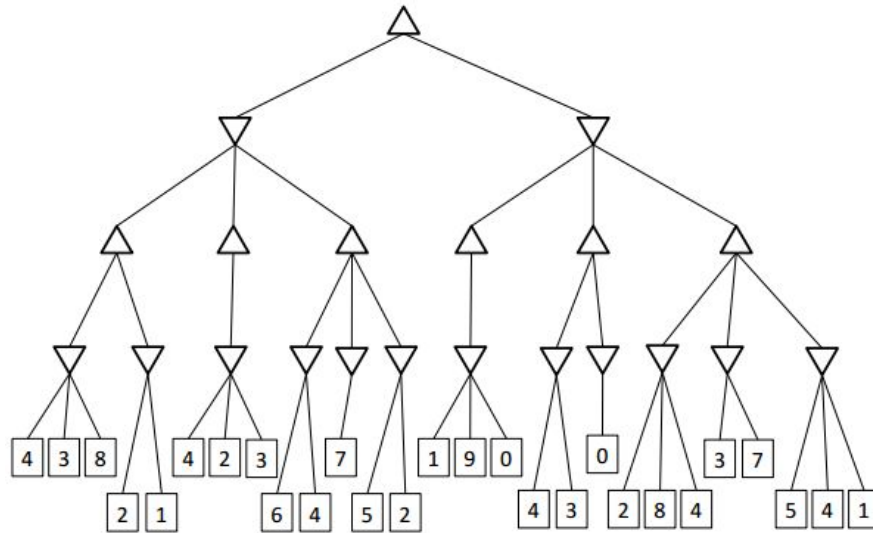
O valor encontrado é 3

2 não é maior ou igual a 3



## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.

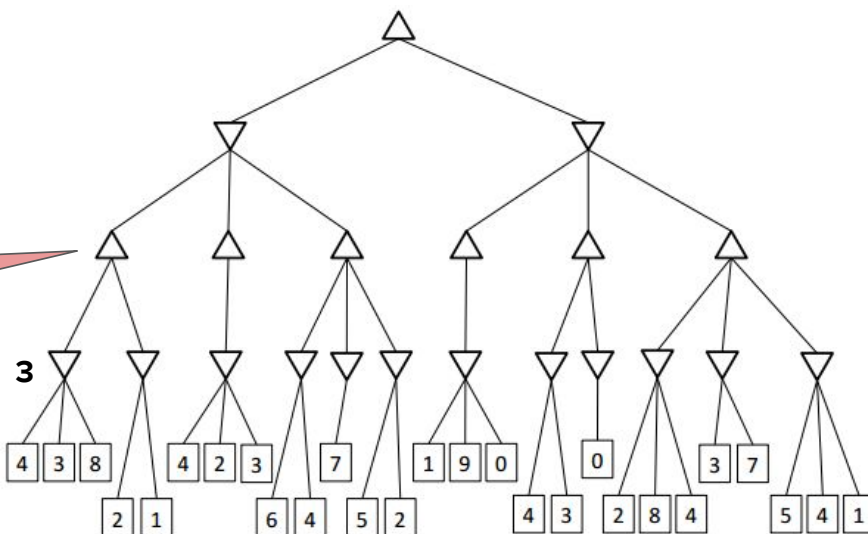


57

## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.

Procurando por um valor  
maior ou igual a 3



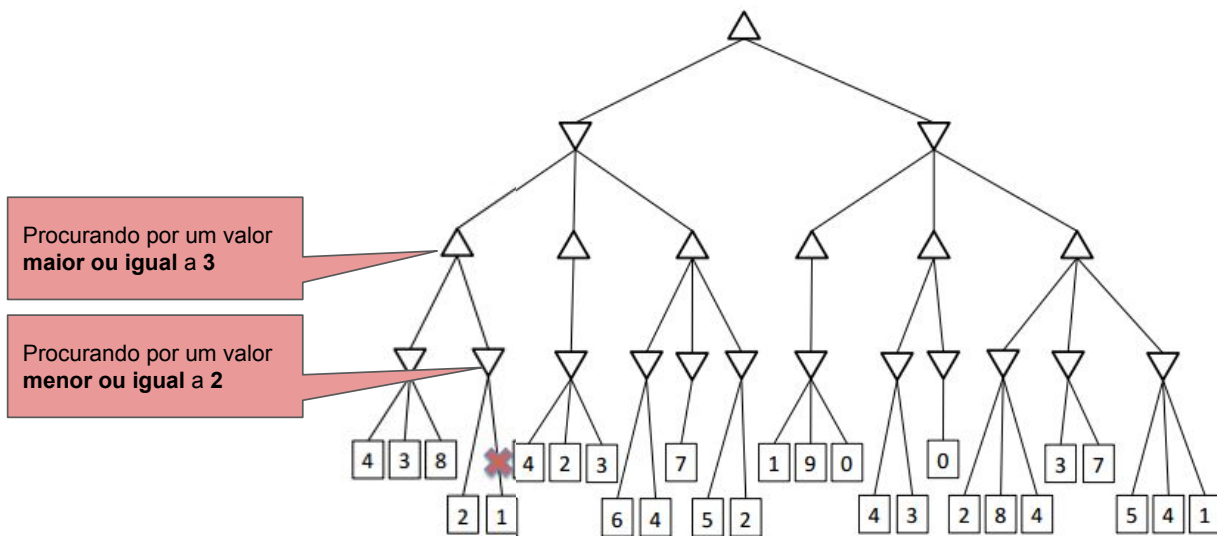
58





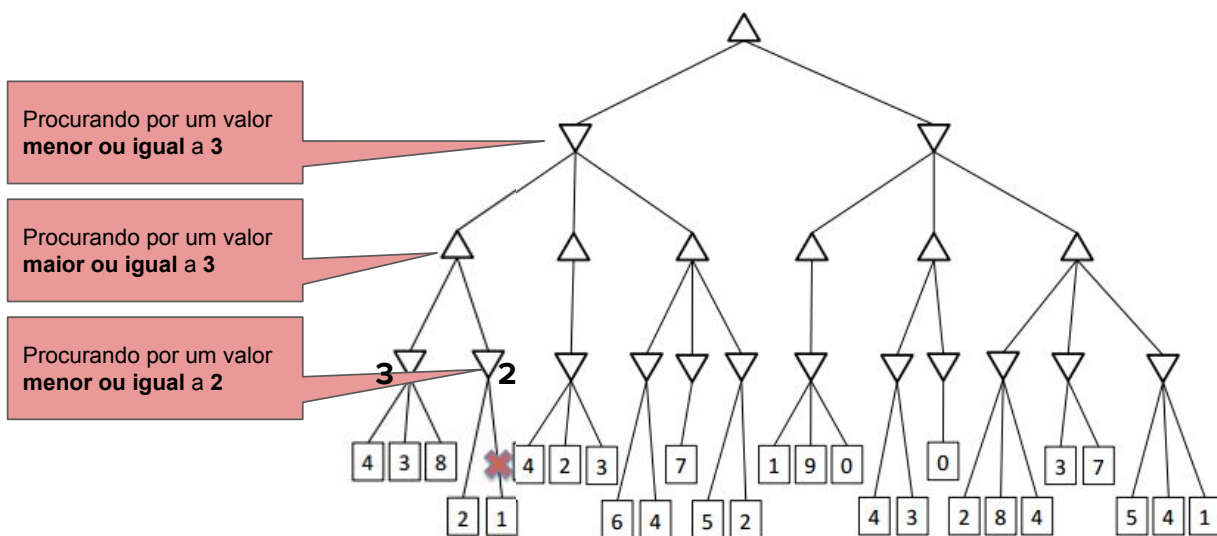
## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.



## Exercício 2

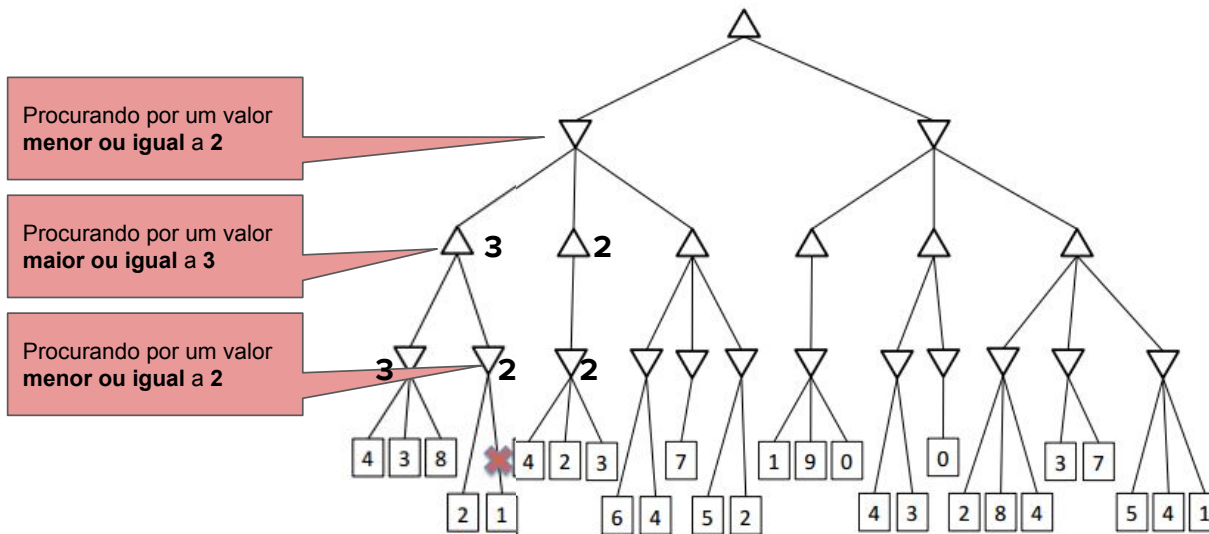
Refaça o exercício anterior usando poda alfa-beta.





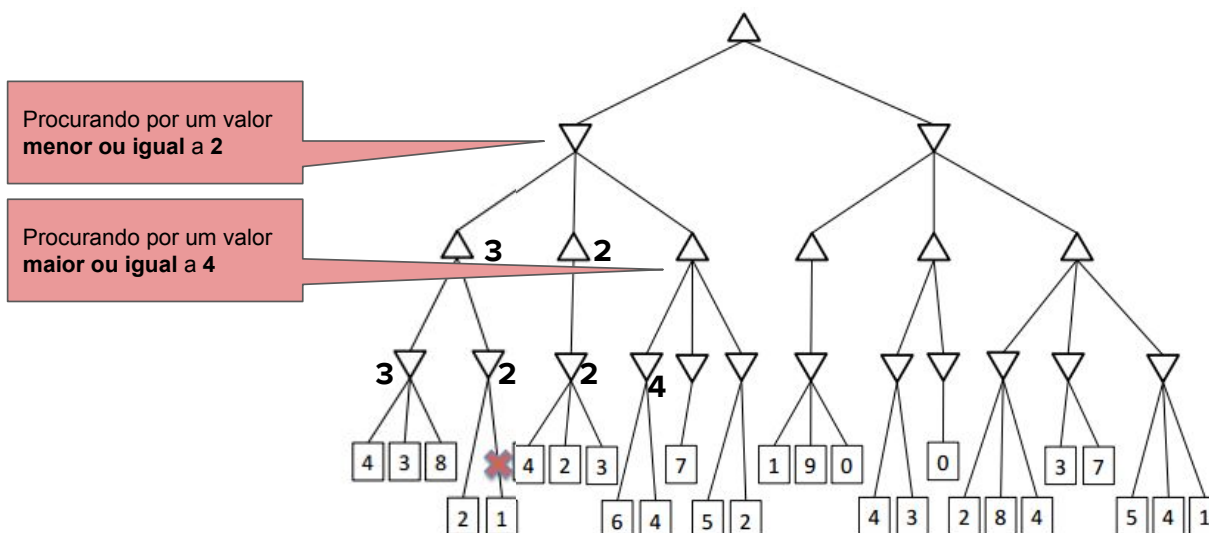
## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.



## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.





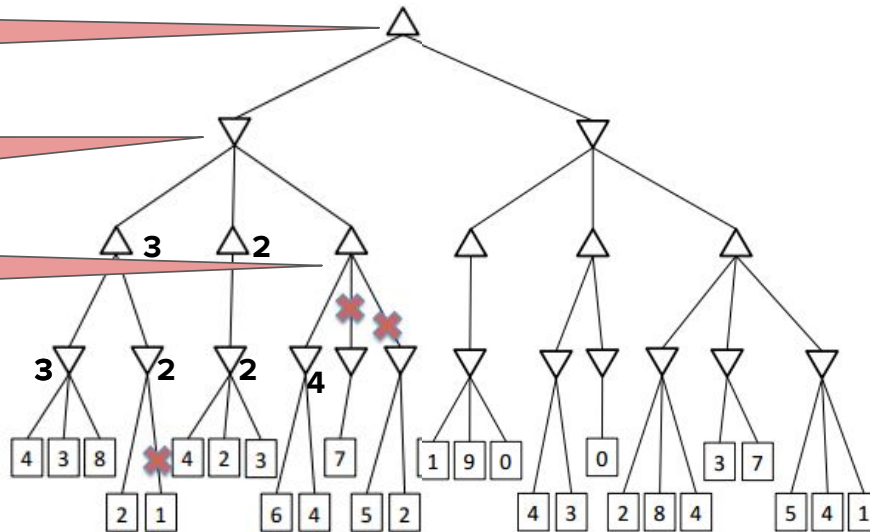
## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.

Procurando por um valor  
maior ou igual a 2

Procurando por um valor  
menor ou igual a 2

Procurando por um valor  
maior ou igual a 4

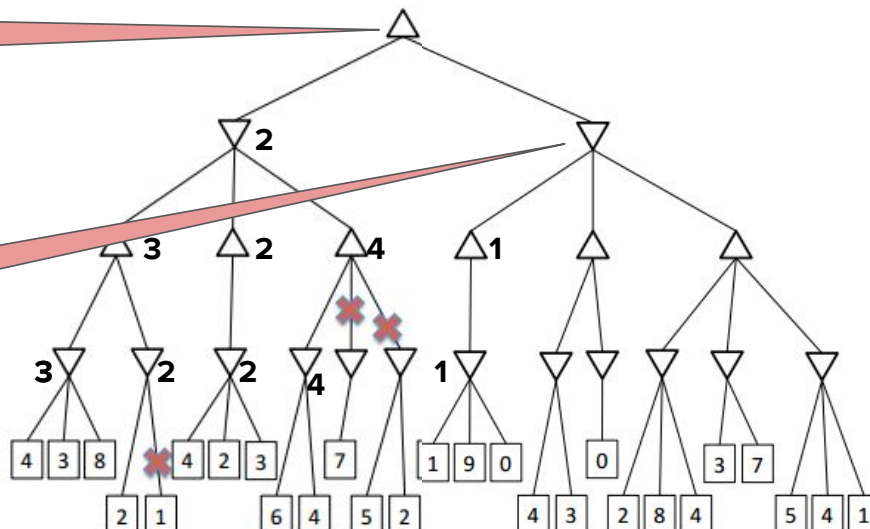


## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.

Procurando por um valor  
maior ou igual a 2

Procurando por um valor  
menor ou igual a 1



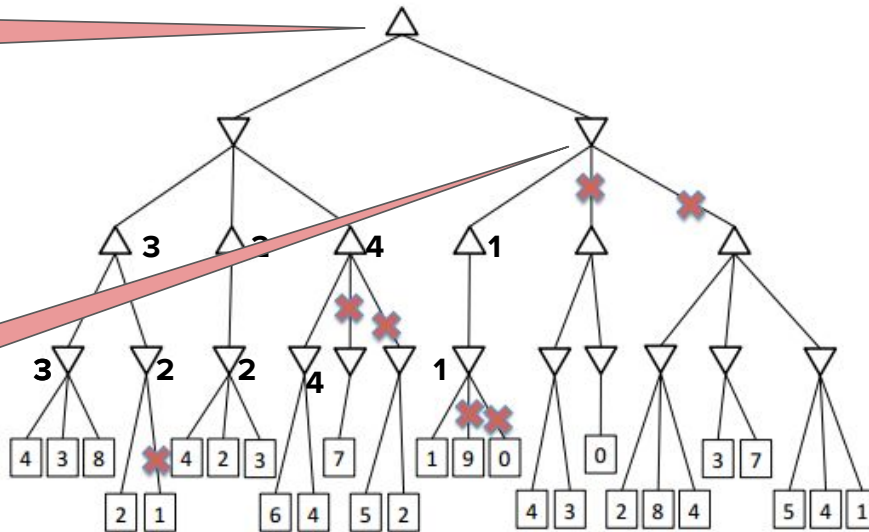


## Exercício 2

Refaça o exercício anterior usando poda alfa-beta.

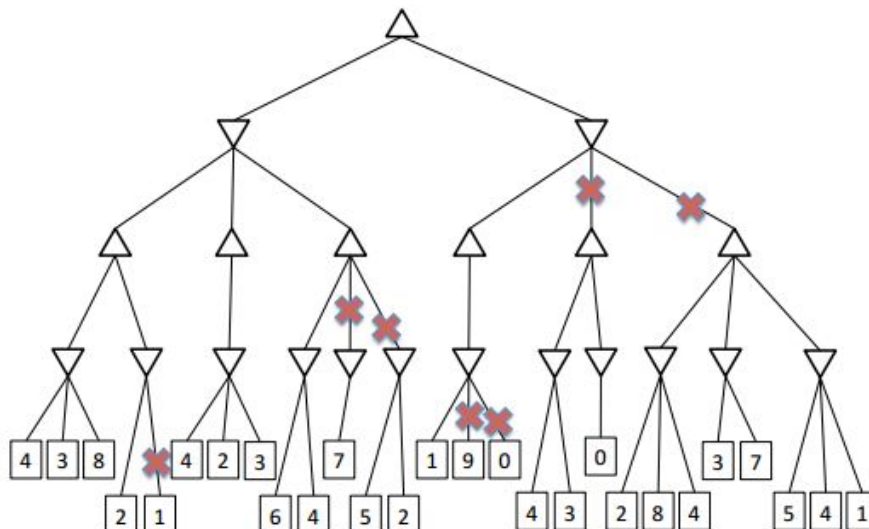
Procurando por um valor maior ou igual a 2

Procurando por um valor menor ou igual a 1



## Exercício 2 - Resposta

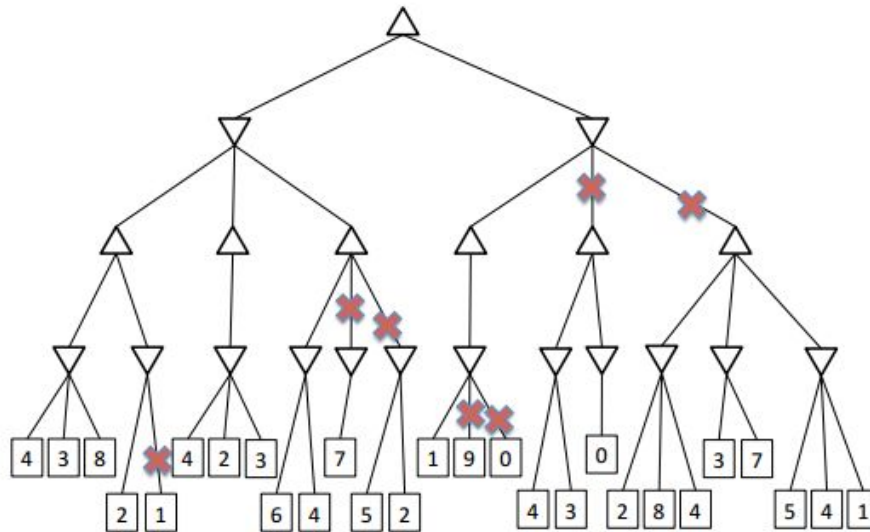
Refaça o exercício anterior usando poda alfa-beta.





## Exercício 2 - Resposta

Refaça o exercício anterior usando poda alfa-beta.



67

## Poda alfa-beta

- Algoritmo

$\alpha$ : MAX's best option on path to root

$\beta$ : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):
    initialize  $v = -\infty$ 
    for each successor of state:
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$ 
        if  $v \geq \beta$  return  $v$ 
         $\alpha = \max(\alpha, v)$ 
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):
    initialize  $v = +\infty$ 
    for each successor of state:
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$ 
        if  $v \leq \alpha$  return  $v$ 
         $\beta = \min(\beta, v)$ 
    return  $v$ 
```

68





## Poda alfa-beta

- Quando perfeitamente ordenado, essa solução pode cair para  $O(b^{m/2})$
- Quando o espaço de busca é muito grande, esta ainda não é uma solução viável!
  - xadrez

## Decisões imperfeitas em RT

- MINIMAX
  - Espaço de busca do jogo inteiro
- Alfa-beta
  - Redução do espaço de busca
  - Ainda pode ter um alto tempo de resposta
  - Profundidade ainda é um problema
- Função de avaliação
  - Heurística
  - Transformação de nós não terminais em nós folhas
  - Substituição de uma função utilidade por uma função heurística + testes de corte





## Decisões imperfeitas em RT

- Função de Avaliação
  - Características do estado
  - Ex. Xadrez
    - Número de peões brancos
    - Posições das rainhas, bispos, reis, ...
  - Exemplo de função de avaliação
    - Uso de experiência
      - 72% dos estados com dois peões vs um peão levam a uma vitória: +1
      - 20% levam a uma derrota: 0
      - 8% a um empate (0.5)



## Decisões imperfeitas em RT

- Função de Avaliação
  - $FA(s) = (0.72 * 1) + (0.2 * 0) + (0.08 * 0.5) = 0.76$
  - Função linear ponderada
  - Função não-linear
    - Bispos têm maior valor no final do jogo



## Decisões imperfeitas em RT

- Busca com corte
- Poda adiantada
  - Busca em feixe
  - Estatística usando probabilidade estimar, com base em experiências passadas, como o valor de um nó em uma dada profundidade provavelmente ficaria fora da janela ( $\alpha$ ,  $\beta$ )

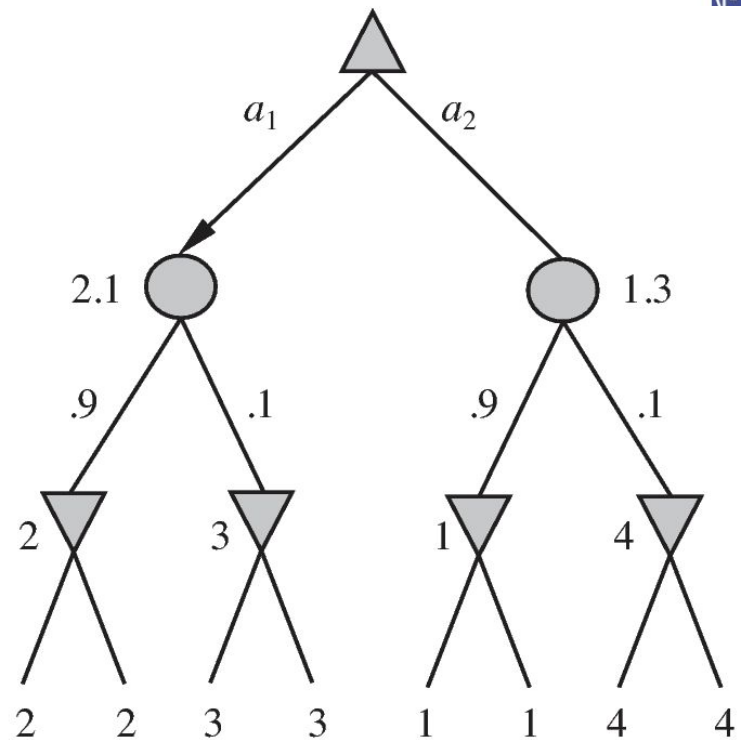
## Jogos Estocásticos

- Nós gerados ao acaso
  - Lançamento de um dado
  - Escolha de uma carta
- Uso de probabilidade



# Jogos Estocásticos

- Nós gerados ao acaso
  - Lançamento de um dado
  - Escolha de uma carta
- Uso de probabilidade



75

## EXPECTIMINIMAX

- Algoritmo

EXPECTIMINIMAX( $n$ )=

UTILIDADE( $n$ )

$\max_{s \in \text{sucessores}(n)} \text{EXPECTIMINIMAX}(s)$

$\min_{s \in \text{sucessores}(n)} \text{EXPECTIMINIMAX}(s)$

$\sum_{s \in \text{sucessores}(n)} P(s) \cdot \text{EXPECTEDMINIMAX}(s)$

Se  $n$  é terminal

Se  $n$  é um nó de MAX

Se  $n$  é um nó de MIN

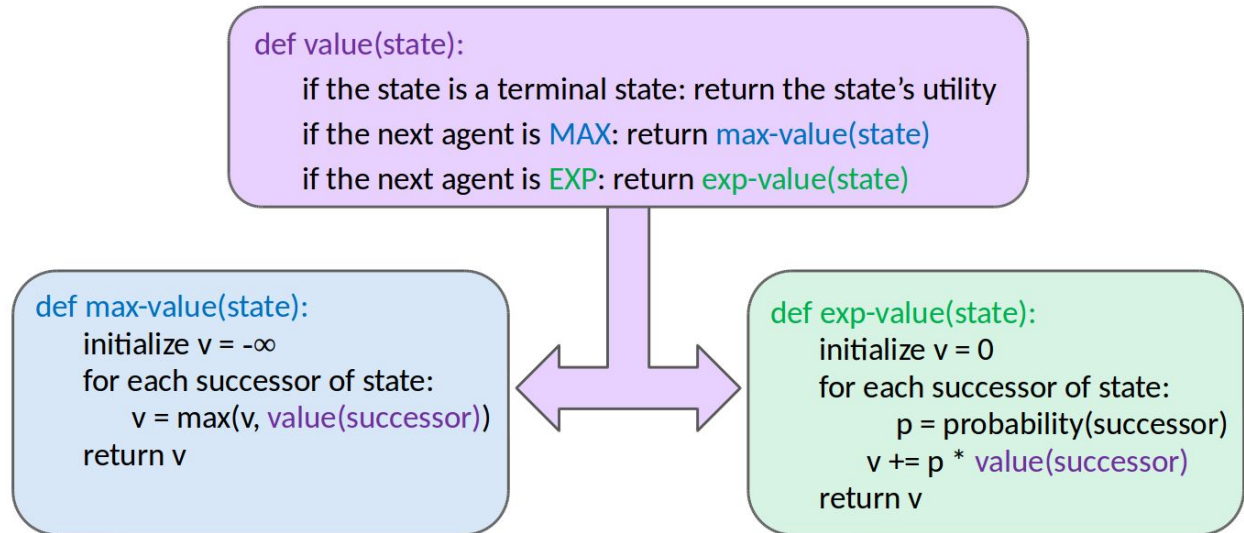
Se  $n$  é um nó de acaso

76



# EXPECTIMAX

- Algoritmo



77

## Jogos Parcialmente Observáveis

- Observabilidade parcial
- Técnicas baseadas na crença agente
- Exemplo:
  - Kriegspiel
  - Jogos de carta



78



## Referência Bibliográfica

- RUSSELL, S.; NORVIG, P. Inteligência Artificial. Editora Campus, 2013, Cap. 05.
- CS 188: Artificial Intelligence, Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley. All CS188 materials are available at <http://ai.berkeley.edu>.